There is a universal Turing machine

The Open Logic Project contributors

openlogic:turing-machines/undecidability:local-snapshot-20261013;
CC BY 4.0

Editorial note. The counted statement is the author's own theorem environment thm:universal-tm (source lines 1434-1443, the \ollabel on line 1434, which the author places on the same line as the environment start); the proof environment begins two lines below the statement and is its own proof.

Editorial note. The proof is an outline that ends by invoking the Church-Turing thesis, as the author says explicitly in its first sentence; the retained package therefore carries the author's own outline and not a machine-level construction, and the card records this so a human can spot-check the level of detail.

Editorial note. The proof contains one internal \olref to the labelled enumerate item that names the instruction-lookup step; that label lies inside the retained proof span, so the reference resolves inside the package and no source pointer is needed.

Editorial note. The retained preamble is the minimal working preamble shipped with the frozen OLP unit (OLP's own style stack needs prooftree, ntheorem, extarrows, mathdots, isomath and subfiles, which the project toolchain does not have). Author text is untouched; only the packaging is the intake line's.

Editorial note. Section and theorem counters are reset to zero so the retained theorem prints with the numbering of the retained front matter rather than continuing the frozen file's count.

Editorial scope. One original result of the Open Logic Project's undecidability unit, counted once: there exists a universal Turing machine.

Allowed standard prerequisites (editorial).

- Turing machine and its index. Turing machines are enumerated by natural numbers in the standard way, and M_e denotes the machine with index e; the index can be written on a tape in unary as a list of blocks encoding the instruction table.. The construction starts by decoding the index e of the machine to be simulated (source lines 1448-1456).
- Encoding of a configuration in unary on the simulating tape. States, tape symbols and head positions are represented as blocks of strokes separated by blanks, with the symbol numbers 1, 2 and 3 for the end marker, the blank and the stroke.. The simulated tape records the contents as blocks of three strokes and the head position as a counter that is extended or shortened (source lines 1456-1465, 1485-1492).

- The Church-Turing thesis. Every effectively computable function is computed by some Turing machine.. The author invokes the thesis at source lines 1447-1448 to conclude that the described simulation really is carried out by a Turing machine, since the machine is only described in outline.

Theorem 0.1. *There is a* universal Turing machine U *which, when started on input* $\langle e, n\rangle$

1. *halts iff* M_e *halts on input* n*, and*
2. *if* M_e *halts with output* m*, so does* U*.*

U *thus computes the function* $f\colon \mathbb{N}\times\mathbb{N} \rightharpoonup \mathbb{N}$ *given by* $f(e,n) = m$ *if* M_e *started on input* n *halts with output* m*, and undefined otherwise.*

Proof. To actually produce U is basically impossible, since it is an extremely complicated machine. But we can describe in outline how it works, and then invoke the Church–Turing thesis. When it starts, U's tape contains a block of e |'s followed by a block of n |'s. It first "decodes" the index e to the right of the input n. This produces a list of numbers (i.e., blocks of |'s separated by ⊔'s) that describes the instructions of machine M_e. U then writes the number of the start state of M_e and the number 1 on the tape to the right of the description of M_e. (Again, these are represented in unary, as blocks of |'s.) Next, it copies the input (block of n |'s) to the right—but it replaces each | by a block of three |'s (remember, the number of the | symbol is 3, 1 being the number of ▷ and 2 being the number of ⊔). At the left end of this sequence of blocks (separated by ⊔ symbols on the tape of U), it writes a single |, the code for ▷.

U now has on its tape: the index e, the number n, the code number of the start state (the "current state"), the number of the initial head position 1 (the "current head position"), and the initial contents of the "tape" (a sequence of blocks of |'s representing the code numbers of the symbols of M_e—the "symbols"—separated by ⊔'s).

It now simulates what M_e would do if started on input n, by doing the following:

1. Find the number k of the "current head position" (at the beginning, that's 1),
2. Move to the kth block in the "tape" to see what the "symbol" there is,
3. Find the instruction matching the current "state" and "symbol,"
4. Move back to the kth block on the "tape" and replace the "symbol" there with the code number of the symbol M_e would write,
5. Move the head to where it records the current "state" and replace the number there with the number of the new state,
6. Move to the place where it records the "tape position" and erase a | or add a | (if the instruction says to move left or right, respectively).
7. Repeat.[1]

[1] We're glossing over some subtle difficulties here. E.g., U may need some extra space when it increases the counter where it keeps track of the "current head position"—in that case it will have to move the entire "tape" to the right.

If M_e started on input n never halts, then U also never halts, so its output is undefined.

If in step 3 it turns out that the description of M_e contains no instruction for the current "state"/"symbol" pair, then M_e would halt. If this happens, U erases the part of its tape to the left of the "tape." For each block of three |'s (representing a | on M_e's tape), it writes a | on the left end of its own tape, and successively erases the "tape." When this is done, U's tape contains a single block of |'s of length m.

If U encounters something other than a block of three |'s on the "tape," it immediately halts. Since U's tape in this case does not contain a single block of |'s, its output is not a natural number, i.e., $f(e, n)$ is undefined in this case. □